WarrantyGraph
Full Project & Codebase Encyclopedia

Complete inventory of the repository — every package, API, pipeline, UI surface, LLMOps control plane, deploy path, tests, and data artifact.

Not session-scoped. This document covers the **entire project** as it exists in the repo (agents, API, graph, ETL, frontend, evals, security, k8s, docs corpus, etc.). Session notes (ontology, Redis, interview Q&A) are only *parts* of this whole.

Stack: Next.js 16 · FastAPI · Neo4j · LangGraph · GraphRAG · FMEA/Bayes · Enterprise ETL · LLMOps

1. Product identity & intent

WarrantyGraph is an **enterprise-style remote diagnostics platform** for appliance warranty support. A user describes a problem in natural language; the system resolves product/asset context, walks a **Neo4j knowledge graph**, ranks failure modes with **deterministic FMEA + Bayesian** scoring, returns steps/parts/provenance, and can escalate or submit a claim. **Core diagnosis does not require an LLM** — optional gateway is off by default.

Facet	Value
Primary UI	frontend/ Next.js :3000
API	api/ FastAPI :8080
Graph	Neo4j Bolt :7687
Mock enterprise	simulation/ :8090
Archived UI	ui-streamlit-archive/
Catalog size	~13 products (10 OEM builders + 3 legacy)

2. Repository map (top-level)

```
agents/ api/ config/ data/ deploy/ docker/ docs/ domain/ evals/
finops/ frontend/ gateway/ graph/ guardrails/ infra/ integrations/
k8s/ models/ monitoring/ observability/ promptops/ prompts/
runtime/ security/ services/ simulation/ tests/ ui-streamlit-archive/
utils/  + Makefile, restart-all.sh, requirements*.txt, .github/
```

3. Layered architecture

Layer	Packages	Responsibility
Experience	frontend, ui-streamlit-archive	Chat, explorer, claims, ops, admin
API	api	HTTP contracts, middleware, admin pipelines
Orchestration	services, agents, integrations	Warranty, LangGraph, claims/cases
Intelligence	graph (non-ETL)	GraphRAG, reliability, trees, parts, viz
Knowledge platform	enterprise_pipeline, populate_graph, OEM catalog	ETL → Neo4j
Cross-cutting	runtime, guardrails, observability, finops, gateway	Scale, safety, cost, LLM, telemetry
Config/domain	config, domain	Settings + typed outcomes
Ops data	utils persistence/lineage/escalation	SQLite + JSONL
Deploy	docker, k8s, deploy, infra, monitoring	Run & observe

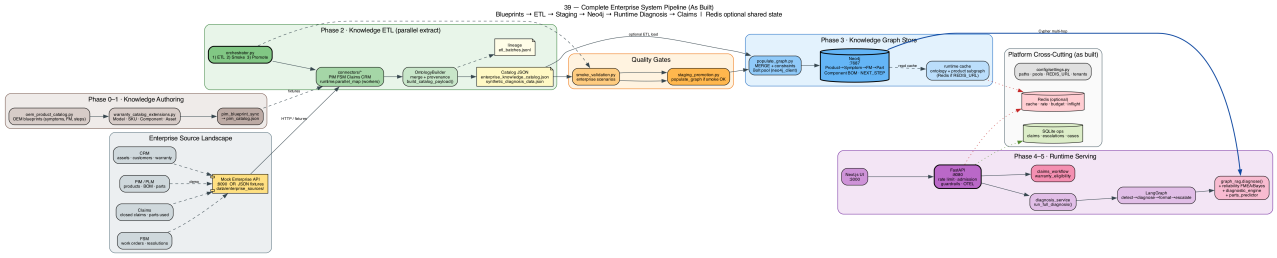

Fig. 1 — Complete system pipeline (as built)

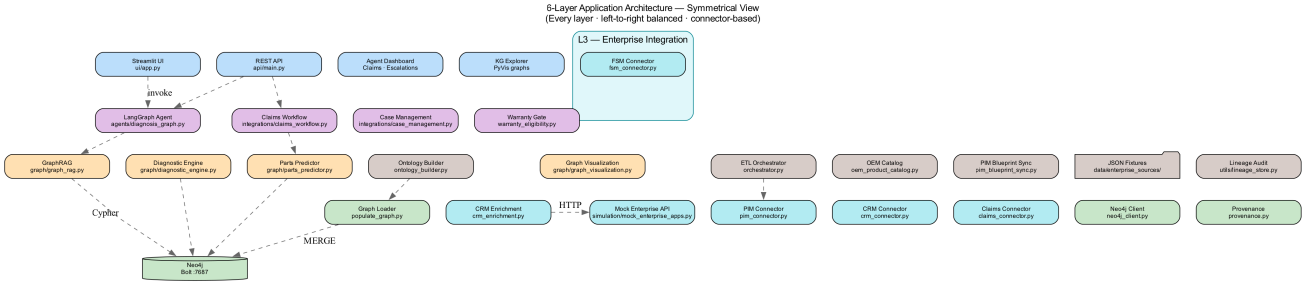

Fig. 2 — Symmetric layer architecture

4. Package encyclopedia (full codebase)

4.1 api/ — REST surface

`main.py` FastAPI app; `schemas.py` request/response models.

Method	Path	Purpose
GET	/health	Liveness + Neo4j + runtime/redis stats
GET	/metrics	Prometheus metrics
GET	/products	List products
POST	/diagnose	Full diagnosis workflow
GET	/graph/ontology	Schema meta-graph (cached)
GET	/graph/product/{id}	Product neighborhood (cached)
GET	/graph/diagnosis-subgraph	Path-focused subgraph
GET/POST/PATCH	/claims*	Claim list/submit/status
GET	/lineage/batches	ETL audit batches
GET	/integrations/status	Connector health
POST/GET	/admin/pipeline/*	dry-run, validate, review, promote, onboard, status

Middleware: X-Request-ID, diagnose rate limit, concurrency admission, CORS, OTEL instrument, exception handler.

4.2 services/ — single business path

`diagnosis_service.run_full_diagnosis`: warranty short-circuit → `agents.run_diagnosis` → optional case handoff. Shared so API/UI rules cannot drift.

4.3 agents/ — LangGraph workflow

- `diagnosis_graph.py` — nodes: detect_product → run_diagnosis → format_response → handle_escalation
- `tools.py` — tool_diagnose, tool_detect_product, list products, steps, rank failures
- Runs without external LLM (graph-native demo mode)

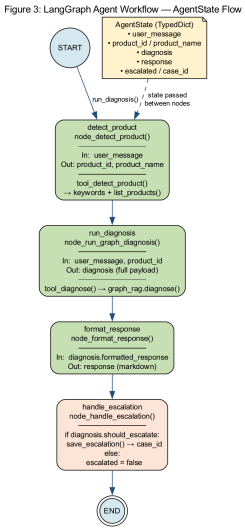

Fig. 3 — LangGraph workflow

4.4 graph/ — intelligence + knowledge load

Module	Responsibility
neo4j_client.py	Bolt driver singleton + connection pool
populate_graph.py	Constraints + MERGE catalog into Neo4j
graph_rag.py	diagnose(), product resolve, match, rank, format, provenance
reliability.py	FMEA S/O/D, RPN, Action Priority, Bayes, dominance, strength labels
diagnostic_engine.py	NEXT_STEP / CONFIRMS / RULES_OUT trees
parts_predictor.py	REQUIRES_PART + BOM + SKU + claim precedent
symptom_retrieval.py	Hybrid lexical + TF-IDF cosine
graph_visualization.py	Ontology/product/diagnosis graph JSON + HTML helpers

Module	Responsibility
knowledge_lineage.py	Per-product knowledge profile
provenance.py	PROV-O aligned source metadata
oem_product_catalog.py	10 OEM product blueprint builders
warranty_catalog_extensions.py	Model/SKU/Component/Asset/Claim extensions
synthetic_data_generator.py	Legacy products + authoritative catalog build
rdf_ontology_export.py	Export Turtle / RDF-XML ontology

4.5 graph/enterprise_pipeline/ — ETL platform

Component	Role
orchestrator.py	Run ETL → smoke → promote in order
connectors/pim|crm|claims|fsm	HTTP mock or JSON fixture extract
connectors/base.py	ConnectorResult + ABC
transformers/ontology_builder.py	Merge sources → validated catalog + provenance
transformers/pim_blueprint_sync.py	OEM blueprints → pim_catalog.json
pipelines/knowledge_etl.py	Pipeline 1 extract/transform/write/load
pipelines/smoke_validation.py	Pipeline 2 scenario gate
pipelines/staging_promotion.py	Pipeline 3 promote if smoke passed
http_client.py	GET/POST JSON helper

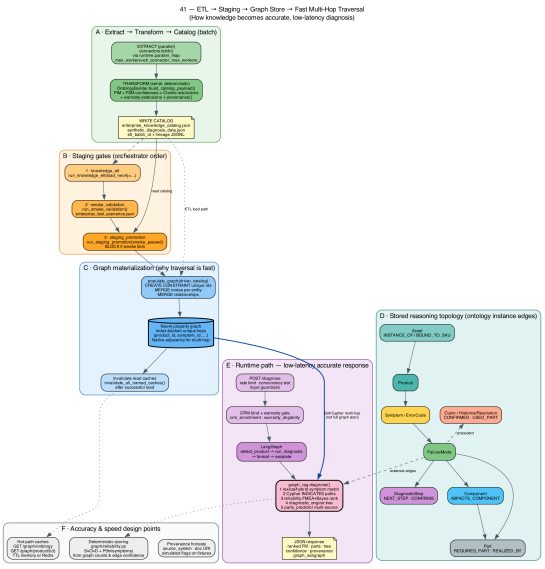

Fig. 4 — ETL → staging → graph traversal

4.6 integrations/ + simulation/

- **crm_enrichment.py** — customer/asset → product/SKU/warranty context
- **warranty_eligibility.py** — coverage decision, parts cost vs policy cap
- **claims_workflow.py** — submit/list/update claims; optional Neo4j Claim MERGE
- **case_management.py** — escalation → simulated CCaaS case
- **simulation/mock_enterprise_apps.py** — mock PIM/CRM/FSM/Claims/Cases on :8090

4.7 domain/ + config/

`domain/models.py`: WarrantyDecision, DiagnosisOutcome. `config/settings.py`: Neo4j, paths, demo flags, Redis, cache TTLs, rate limits, LLM keys, OTEL, FinOps budget, tenant, pools, admin token. Fails fast on default password outside demo_mode.

4.8 runtime/ — scale primitives

- cache.py — in-process TTL or RedisTtlCache; named caches; invalidate after ETL
- redis_client.py — optional shared state; health for /health
- concurrency.py — parallel_map for connector I/O
- concurrency_limit.py — max concurrent diagnoses (503 when saturated)
- partitioning.py — tenant/product/batch keys + batch_items

4.9 guardrails/ + observability/ + finops/ + gateway/

Package	Modules / role
guardrails	input (injection), output (PII/length), pipeline, action allowlist, rate_limit
observability	JSON logging + request_id, Prometheus metrics, OTEL tracing, redaction
finops	DailyCostBudget circuit breaker (memory or Redis)
gateway	ModelGateway: alias→provider, retry/fallback, meter, budget check
promptops + prompts	Versioned YAML prompts (e.g. diagnosis-rewriter/v1)
models/	registry.yaml pinned model aliases

4.10 utils/

- persistence.py — SQLite OperationalStore (escalations, cases, claims)
- escalation_store.py / lineage_store.py — agent queue + ETL batch audit
- diagnosis_display.py — executive formatting, mermaid journeys, grounding tables
- connector_status.py — integration_status() for UI/ops
- logger.py — get_logger helper

4.11 frontend/ — Next.js 16 experience

Path	Role
app/page.tsx	Main UI: diagnosis chat, knowledge explorer, claims, ops, admin
app/layout.tsx, providers.tsx	Shell + React Query providers
lib/api.ts	Client for all backend routes
lib/types.ts	Frontend types
globals.css	Dark/light design tokens

Features: natural-language chat; recommendation strength badges; 3-tile confidence (posterior / graph link / text match); provenance trail; React Flow explorer with path highlight; agent cases; ETL lineage; admin pipeline actions.

4.12 evals/ + tests/

evals/: run_eval.py gate; golden/smoke.jsonl; safety/injection.jsonl; thresholds.yaml. **tests/**: API, diagnosis, reliability, symptom retrieval, product resolution, services, guardrails, observability, OEM catalog, warranty ontology, pipeline integration, enterprise scenarios, graph viz, persistence, runtime/Redis, RDF export, e2e flow.

4.13 data/

Artifact	Purpose
synthetic_diagnosis_data.json	Primary runtime catalog (often ETL output)
enterprise_knowledge_catalog.json	Full enterprise catalog payload
enterprise_sources/*.json	PIM/CRM/FSM/claims fixtures
provenance_manifest.json	Default provenance by entity type
lineage/etl_batches.jsonl	Batch audit log
diagnostics.db	SQLite ops store
escalations.json / simulated_cases.json	Supporting/legacy case data

4.14 Deploy · infra · security · monitoring

- **docker/** — Dockerfile.api|etl|frontend|mock|ui; compose observability + redis
- **k8s/base** — API, UI, mock, Neo4j StatefulSet, ETL CronJob, ingress, PVC, configmap
- **k8s/overlays** — staging / prod
- **deploy/rollouts** — Argo Rollouts + Flagger canary manifests
- **infra/terraform** — IaC modules/README
- **monitoring/** — alerts, Grafana dashboard, OTEL collector, SLO rules
- **security/** — threat-model.md, owasp-llm-mapping.md
- **docs/governance/** — classification, retention, DPIA
- **.github/workflows** — ci.yml, cd.yml, eval-nightly.yml

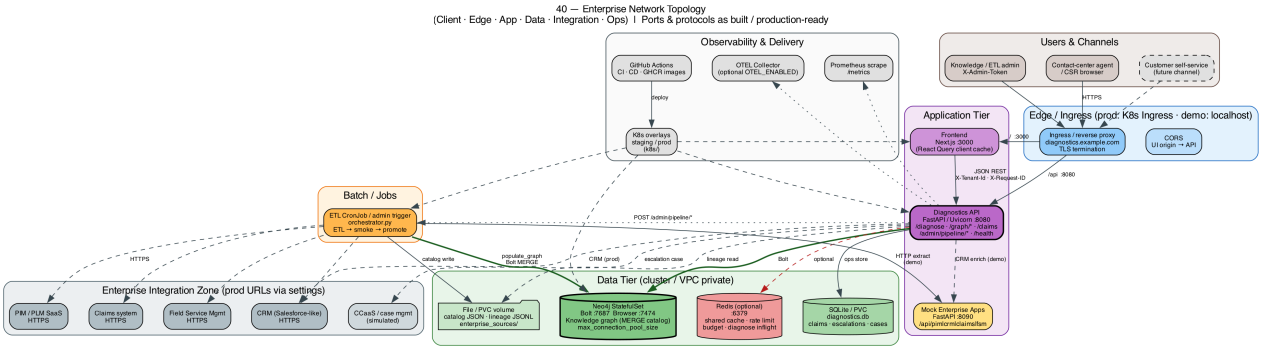

Fig. 5 — Enterprise network topology

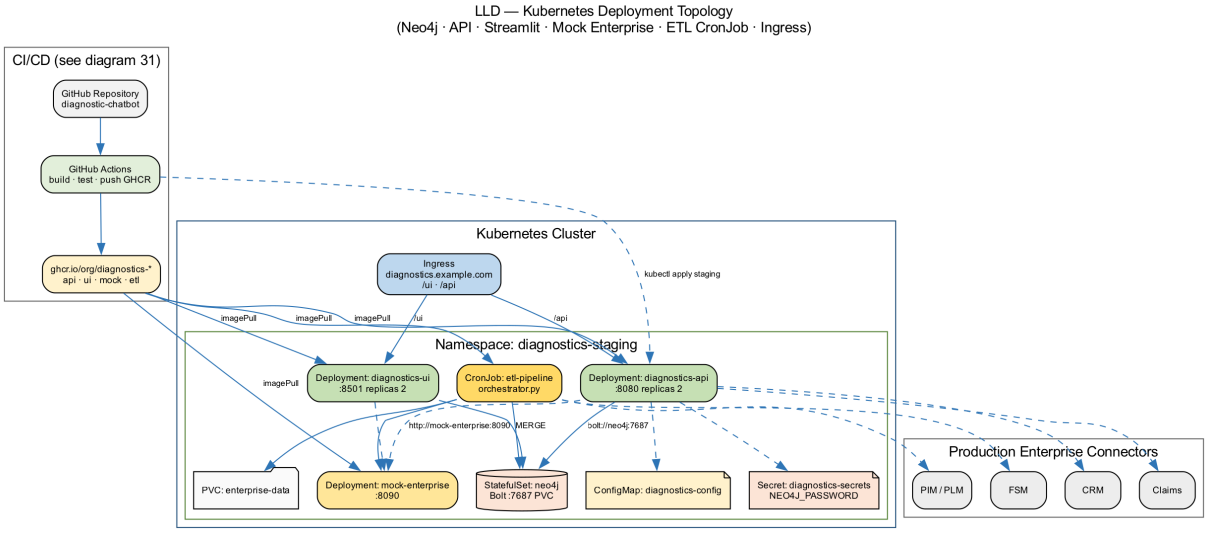

Fig. 6 — K8s deployment topology

4.15 Documentation corpus (full, not only recent)

- docs/01–14 .docx — architecture, GraphRAG, Cypher, roadmaps, demos, methodology
- docs/15–18 .md — ontology/RDF, runtime, landscape, **this full encyclopedia**
- docs/c4 — Structurizr DSL + C4 diagrams
- docs/graphviz 01–41 — pipeline, ERD, reliability, topology, etc.
- docs/llmops-handbook 00–21 + PDFs + implementation playbook
- docs/interview — interview mastery PDF (persona Q&A)
- docs/full-project — this encyclopedia PDF
- docs/runbooks — cost spike, latency, PII, injection, outage, quality, RAG stale
- docs/model-cards/system-card.md

4b. Indexes & constraints (What / Where / When / How / Why)

Companion: docs/19-Indexes-Constraints-and-Lookup-Performance.md

	Neo4j	SQLite ops
What	UNIQUE on each entity *_id (+ unique index)	PK + status indexes
Where	populate_graph.create_constraints	utils/persistence.py
When	Every graph load/promote	First DB open
How	CREATE CONSTRAINT IF NOT EXISTS … IS UNIQUE	CREATE INDEX idx_*_status
Why	Id seek + no duplicates + safe MERGE	Filter agent queues

Not used: Neo4j full-text/vector indexes. Symptom text is hybrid TF-IDF after product resolve. Runtime path: MATCH (p:Product {product_id:$pid}) uses constraint-backed index, then walks edges.

Universal WWWH lens (use for every subsystem)

Topic	What	Where	When
Diagnosis	Rank FM + parts	api→service→graph_rag	Each POST /diagnose
FMEA/Bayes	Risk + posteriors	reliability.py	During FM ranking
ETL	Catalog + graph	enterprise_pipeline	Batch/admin
Redis opt.	Shared multi-pod state	runtime/	If REDIS_URL set

5. Knowledge model (ontology in Neo4j)

Nodes: Product, Model, SKU, Asset, Symptom, ErrorCode, FailureMode, DiagnosticStep, Component, Part, Claim, HistoricalResolution, WarrantyPolicy.

Relationships: HAS_MODEL, HAS_SKU, INSTANCE_OF, BOUND_TO_SKU, HAS_SYMPTOM, HAS_ERROR_CODE, CAN_HAVE, INDICATES, HAS_DIAGNOSTIC_STEP, CONFIRMS, RULES_OUT, NEXT_STEP, IMPACTS_COMPONENT, REALIZED_BY, REQUIRES_PART, COMPATIBLE_WITH, CONFIRMED, USED_PART, FOR_PRODUCT, COVERED_BY.

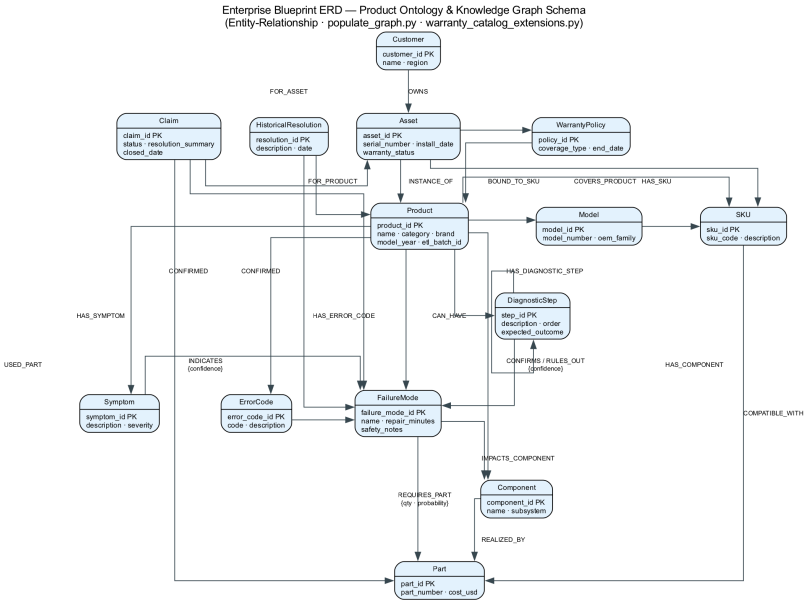

Fig. 7 — Enterprise blueprint ERD

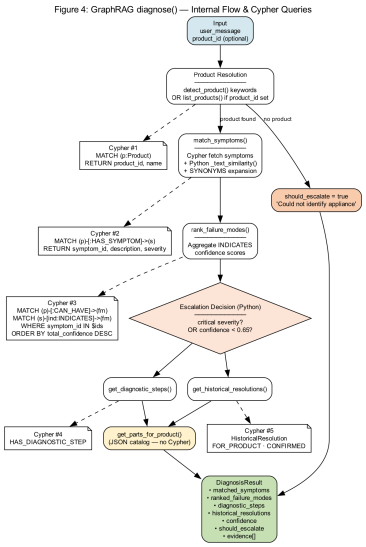

Fig. 8 — GraphRAG diagnosis flow

6. End-to-end sequences

6.1 Online diagnosis

```
POST /diagnose
  rate limit + concurrency slot + guard_request
  CRM enrich + warranty gate
  run_full_diagnosis → LangGraph
    detect product → graph_rag.diagnose
      match symptoms/codes → rank FM (reliability)
      diagnostic tree → parts_predictor → format + provenance
    escalate? → escalation_store / case_management
  validate_output → DiagnoseResponse
```

6.2 Batch knowledge

```
orchestrator /admin/pipeline/*
  knowledge_etl: parallel_map(fetch) → OntologyBuilder → JSON + lineage
                 optional populate_graph + cache invalidate
  smoke_validation (block promote on fail)
  staging_promotion → populate_graph MERGE
```

7. Configuration surface

Groups in settings: Neo4j + pool; data paths; demo/fixture/mock flags; connector URLs; API host/port/admin token; diagnosis thresholds (escalation, symptom score, ambiguity); cache TTLs + ETL workers; Redis URL/prefix; max concurrent diagnoses; rate limit; PII/length; OTEL/Prometheus; LLM enablement + keys; daily budget; default tenant.

8. Product catalog (13)

OEM builders (Samsung/LG/Whirlpool/Bosch/GE washers, dishwashers, dryer, range, fridge, microwave) plus legacy wm-001, dw-001, mw-001. Public-doc patterns; representative service BOM — not secret SKUs.

9. Local runbook

```
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python graph/populate_graph.py
uvicorn api.main:app --port 8080
cd frontend && npm install && npm run dev
# optional: python -m simulation.mock_enterprise_apps
# optional: REDIS_URL=redis://localhost:6379/0
python -m graph.enterprise_pipeline.orchestrator
./restart-all.sh
```

10. Quality gates

- pytest full suite (pre-push hook with venv)
- ruff lint/format (pre-commit)
- evals/run_eval.py golden + safety thresholds
- CI: tests, supply-chain audit, diagram render
- Nightly eval workflow; CD image publish

11. Security posture (honest)

Control	Status
Input injection guardrails	Implemented
Output PII redaction	Implemented
Action allowlist / HITL hooks	Implemented
Rate limit / admission control	Implemented (memory or Redis)
Admin token for pipelines	Optional setting
End-user OIDC/JWT	Not productized (demo open)
Live SAP/SFDC	Mock/fixtures by default; connectors pluggable
Fixture honesty	Provenance / simulated labels

12. Where is X? (quick index)

Need	Location
HTTP entry	api/main.py
Business rules once	services/diagnosis_service.py
Agent steps	agents/diagnosis_graph.py
Diagnosis brain	graph/graph_rag.py
Scoring math	graph/reliability.py
Parts ranking	graph/parts_predictor.py
Diagnostic trees	graph/diagnostic_engine.py
Load Neo4j	graph/populate_graph.py
ETL spine	graph/enterprise_pipeline/
OEM content	graph/oem_product_catalog.py
Settings	config/settings.py
UI API client	frontend/lib/api.ts
Mock backends	simulation/mock_enterprise_apps.py
Cache/Redis	runtime/
Guardrails	guardrails/
Metrics/traces	observability/
Optional LLM	gateway/ + models/registry.yaml
Eval gate	evals/run_eval.py
Kubernetes	k8s/
Diagrams	docs/graphviz/ (esp. 34–41), docs/c4/

13. Explicit gaps / non-goals

- Not full multi-tenant SaaS with OIDC out of the box
- Not live SAP/Salesforce by default (mock + fixtures)
- Not vector-DB-first RAG (graph-first; hybrid lexical optional)
- LLM not required for core path
- Future: async ETL workers, Neo4j HA, Postgres ops DB, claim→learning loop

14. How this relates to other docs

Document	Scope
This encyclopedia (18 + PDF)	Entire codebase & project
docs/15–17	Ontology/RDF, runtime scale, landscape
docs/interview/*	Interview Q&A style prep
README.md	Quick start & product overview
PIPELINE-AND-MODULE-GUIDE.md	Phase 0–5 blueprint→claim
llmops-handbook/*	LLMOps disciplines depth

Editable source of truth: docs/18-FULL-PROJECT-CODEBASE-ENCYCLOPEDIA.md · Regenerate PDF: python docs/full-project/generate_full_project_pdf.py